

13장 동적 메모리 관리와 표준 함수

1. 동적 메모리 관리

- 메모리 할당 방식:
 - 정적 메모리 할당 (Static Memory Allocation): 프로그램을 작성할 때(컴파일 시) 사용할 메모리의 필요량을 미리 지정하여 할당하는 방식입니다.
 - 동적 메모리 할당 (Dynamic Memory Allocation): 프로그램 실행 중에 사용자의 요구에 따라 유동적으로 메모리 할당을 컴퓨터에 요구하는 방식입니다. 이를 통해 메모리 공간의 낭비를 막고 효율적으로 자원을 사용할 수 있습니다.
- 변수 종류에 따른 메모리 할당 영역:
 - 프로그램(Text) 영역: 기계어 코드 등 프로그램의 실행 명령이 저장됩니다.
 - 데이터(Data) 영역: 상수, 전역 변수, 정적 변수 등이 할당되며 프로그램 종료 시 소멸됩니다.
 - 힙(Heap) 영역: 동적 메모리 할당 함수에 의해 실행 시 크기가 결정되어 할당되는 영역입니다.
 - 스택(Stack) 영역: 매개 변수, 지역 변수, 리턴 값 등 함수 호출에 필요한 데이터가 일시적으로 보관됩니다.

2. 동적 메모리 관리 함수 (<stdlib.h>)

동적 메모리는 프로그램 실행 중에 필요한 만큼 메모리를 할당받아 사용하는 방식으로, **힙(Heap) 영역**에 저장됩니다. 프로그램이 종료되기 전까지 직접 메모리를 해제(`free()`)해야 하며, 그렇지 않으면 ****메모리 누수(Memory Leak)****가 발생합니다.

`malloc(size)`

지정한 `size` 바이트만큼 메모리를 할당하고 시작 주소를 `void *` 형태로 반환합니다.

- 메모리의 초기값은 쓰레기값(**Garbage Value**)입니다.
- 할당에 실패하면 `NULL`을 반환합니다.

함수 원형

```
void *malloc(size_t size);
```

사용 예시

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr;

    arr = (int *)malloc(5 * sizeof(int));
```

```
if (arr == NULL) {
    printf("메모리 할당 실패\n");
    return 1;
}

for (int i = 0; i < 5; i++) {
    arr[i] = (i + 1) * 10;
}

for (int i = 0; i < 5; i++) {
    printf("%d ", arr[i]);
}

free(arr);

return 0;
}
```

실행 결과

```
10 20 30 40 50
```

calloc(num, size)

size 크기의 데이터를 **num**개 저장할 수 있는 메모리를 할당합니다.

- 모든 메모리가 **0**으로 초기화됩니다.
- 배열을 만들 때 자주 사용됩니다.

함수 원형

```
void *calloc(size_t num, size_t size);
```

사용 예시

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr;

    arr = (int *)calloc(5, sizeof(int));

    if (arr == NULL) {
        printf("메모리 할당 실패\n");
    }
}
```

```
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }

    free(arr);

    return 0;
}
```

실행 결과

```
0 0 0 0 0
```

realloc(pointer, size)

이미 할당된 메모리의 크기를 변경합니다.

- 메모리를 늘리거나 줄일 수 있습니다.
- 기존 데이터는 가능한 한 유지됩니다.
- 실패하면 **NULL**을 반환하며, 기존 메모리는 그대로 유지됩니다.

함수 원형

```
void *realloc(void *pointer, size_t size);
```

사용 예시

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr;

    arr = (int *)malloc(3 * sizeof(int));

    for (int i = 0; i < 3; i++) {
        arr[i] = i + 1;
    }

    arr = (int *)realloc(arr, 5 * sizeof(int));

    arr[3] = 4;
```

```
    arr[4] = 5;

    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }

    free(arr);

    return 0;
}
```

실행 결과

```
1 2 3 4 5
```

free(ptr)

동적으로 할당받은 메모리를 운영체제에 반환합니다.

- `malloc()`, `calloc()`, `realloc()`으로 할당받은 메모리는 반드시 `free()`로 해제해야 합니다.
- 해제하지 않으면 **메모리 누수(Memory Leak)**가 발생합니다.

함수 원형

```
void free(void *ptr);
```

사용 예시

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *num;

    num = (int *)malloc(sizeof(int));

    *num = 100;

    printf("%d\n", *num);

    free(num);
    num = NULL;      // 안전하게 NULL로 초기화
}
```

```
    return 0;
}
```

malloc()과 calloc()의 차이

항목	malloc()	calloc()
메모리 초기화	하지 않음 (쓰레기값)	0으로 초기화
매개변수	malloc(size)	calloc(num, size)
사용 목적	일반적인 메모리 할당	배열 생성 시 주로 사용

realloc() 사용 시 주의사항

기존 포인터에 바로 대입하면, 메모리 재할당이 실패했을 때 기존 메모리 주소를 잃어버릴 수 있습니다.

권장 방법

```
int *temp;

temp = realloc(arr, 10 * sizeof(int));

if (temp != NULL) {
    arr = temp;
}
```

free() 사용 시 주의사항

메모리를 해제한 후에는 **댕글링 포인터(Dangling Pointer)**를 방지하기 위해 **NULL**을 대입하는 것이 좋습니다.

```
free(arr);
arr = NULL;
```

핵심 요약

함수	기능	초기화 여부
malloc(size)	메모리 할당	❌ 쓰레기값
calloc(num, size)	배열 형태 메모리 할당	✅ 0으로 초기화
realloc(ptr, size)	메모리 크기 변경	기존 데이터 유지(가능한 경우)
free(ptr)	메모리 해제	해당 없음

시험에서 자주 나오는 포인트

- `malloc()`은 초기화하지 않는다(쓰레기값).
- `calloc()`은 0으로 초기화한다.
- `realloc()`은 메모리 크기를 변경하며 기존 데이터를 최대한 유지한다.
- `free()`를 호출하지 않으면 **메모리 누수(Memory Leak)**가 발생한다.
- `free()` 후에는 `ptr = NULL;`을 하는 습관을 들이는 것이 좋다.

3. 수학연산 관련 함수 (<math.h>)

복잡한 수학 계산을 지원하는 함수들입니다.

- 삼각 함수: `sin(x)`, `cos(x)`, `tan(x)` (인수 x는 라디안 단위 적용).
- 지수 및 로그 함수: `exp(x)` (e^x), `log(x)` (자연로그), `log10(x)` (상용로그).
- 제곱 및 제곱근: `pow(x, y)` (x^y), `sqrt(x)` ($\sqrt{x}$).
- 소수점 처리 및 분할:
 - `ceil(x)`: 소수점 이하 올림 처리.
 - `floor(x)`: 소수점 이하 버림 처리.
 - `fmod(x, y)`: 두 실수 Ha눓셈(x/y)의 나머지 반환.
 - `modf(x, &intptr)`: 실수를 소수부(반환값)와 정수부(`intptr`에 저장)로 나누어 분할.

4. 문자열-수치 변환 함수 (<stdlib.h>)

문자열로 이루어진 숫자를 실제 수치 계산이 가능한 데이터형으로 변환합니다. 반대로 수치 데이터를 문자열로 변환할 때는 `sprintf_s()` 함수를 사용합니다.

- `atoi(str)`: 문자열을 `int`형 정수로 변환.
- `atol(str)`: 문자열을 `long int`형 정수로 변환.
- `atof(str)`: 문자열을 `double`형 부동소수점 실수로 변환.

5. 문자 평가 함수 (<ctype.h>)

하나의 문자가 특정 조건(숫자, 대문자 등)에 부합하는지 검사하여, 참일 경우 0이 아닌 값을 반환하고 거짓일 경우 0을 반환합니다.

- `isalnum(c)`: 알파벳이나 숫자이면 참.
- `isalpha(c)`: 알파벳이면 참.
- `isdigit(c)`: 10진수 숫자(0~9)이면 참.
- `isxdigit(c)`: 16진수 숫자(0~9, a~f, A~F)이면 참.
- `isspace(c)`: 공백이나 탭 등의 공백 문자이면 참.
- `isupper(c)` / `islower(c)`: 각각 대문자 / 소문자이면 참.
- `isprint(c)`: 화면에 인쇄(출력) 가능한 문자이면 참.

6. 문자 변환 함수 (<ctype.h>)

- `tolower(c)`: 전달된 문자가 대문자이면 소문자로 변환 (아닐 경우 그대로 반환).
- `toupper(c)`: 전달된 문자가 소문자이면 대문자로 변환 (아닐 경우 그대로 반환).
- `toascii(c)`: 해당 문자의 하위 7비트만 취하여 ASCII 형식(0~127 범위)으로 변환.